Postman Variables

Variables in Postman allow you to store and reuse values in your requests and scripts. They make your collections more flexible, maintainable, and environment-independent. This guide covers all variable scopes in Postman, with explanations and examples for each type.

Variable Scopes in Postman

Postman offers several scopes for variables, listed here in order of precedence (highest to lowest):

1. **Local Variables**: Temporary variables that only exist during the execution of a request or collection run
2. **Data Variables**: Values from data files used in collection runs
3. **Environment Variables**: Values specific to an environment (e.g., development, testing, production)
4. **Collection Variables**: Values shared across all requests in a collection
5. **Global Variables**: Values available across all collections

When Postman encounters a variable reference, it resolves it by looking through these scopes in order until it finds a match.

1. Global Variables

Global variables have the widest scope in Postman and are accessible across all collections.

Characteristics

- Accessible in all collections and environments
- Persist until manually cleared or until the Postman application is closed
- Useful for values that rarely change and are used across multiple collections

Setting Global Variables

Via the UI

1. Click on the "Environments" dropdown in the top right corner
2. Select "Globals"
3. Add key-value pairs and save

Via Scripts

```
// Set a global variable
pm.globals.set("apiKey", "abc123xyz456");

// Set multiple global variables at once
pm.globals.set({
    "baseUrl": "https://api.naveenautomationlabs.com",
    "version": "v1"
});
```

Getting Global Variables

In Requests

Use the double curly brace syntax:

```
https://{{baseUrl}}/users
```

In Scripts

```
// Get a global variable
const apiKey = pm.globals.get("apiKey");
console.log("API Key:", apiKey);

// Check if a global variable exists
if (pm.globals.has("apiKey")) {
    console.log("API Key is set");
}
```

Unset Global Variables

```
// Remove a single global variable
pm.globals.unset("tempToken");

// Clear all global variables
pm.globals.clear();
```

2. Collection Variables

Collection variables are available throughout a specific collection.

Characteristics

- Scoped to a particular collection
- Available to all requests within that collection
- Persist with the collection
- Useful for values that are specific to a collection but consistent across environments

Setting Collection Variables

Via the UI

1. Select the collection in the sidebar

2. Click on the "Variables" tab
3. Add key-value pairs and save

Via Scripts

```
// Set a collection variable
pm.collectionVariables.set("resourceId", "user123");

// Set multiple collection variables
pm.collectionVariables.set({
    "endpoint": "/users",
    "requestTimeout": 5000
});
```

Getting Collection Variables

In Requests

```
https://api.example.com{{endpoint}}?timeout={{requestTimeout}}
```

In Scripts

```
// Get a collection variable
const resourceId = pm.collectionVariables.get("resourceId");
console.log("Resource ID:", resourceId);

// Check if a collection variable exists
if (pm.collectionVariables.has("endpoint")) {
    console.log("Endpoint is defined");
}
```

Unset Collection Variables

```
// Remove a single collection variable
pm.collectionVariables.unset("resourceId");

// Clear all collection variables
pm.collectionVariables.clear();
```

3. Environment Variables

Environment variables are associated with specific environments (like development, testing, staging, production).

Characteristics

- Connected to a specific environment
- Allow the same collection to work with different backends/configurations
- Only active when the associated environment is selected
- Useful for environment-specific values (URLs, credentials, etc.)

Setting Up Environments

Via the UI

1. Click on the "Environments" dropdown in the top right
2. Click "Add" to create a new environment
3. Name your environment (e.g., "Development", "Production")
4. Add key-value pairs and save

Via Scripts

```
// Set an environment variable
pm.environment.set("baseUrl", "https://dev-api.example.com");

// Set multiple environment variables
pm.environment.set({
    "username": "devuser",
    "password": "devpass",
    "timeout": 10000
});
```

Getting Environment Variables

In Requests

```
https://{{baseUrl}}/auth?username={{username}}&password={{password}}
```

In Scripts

```
// Get an environment variable
const baseUrl = pm.environment.get("baseUrl");
console.log("Base URL:", baseUrl);

// Check if an environment variable exists
if (pm.environment.has("username")) {
    console.log("Username is set in this environment");
}
```

Unset Environment Variables

```
// Remove a single environment variable
pm.environment.unset("tempToken");

// Clear all environment variables
pm.environment.clear();
```

4. Data Variables

Data variables come from external data files (CSV or JSON) and are used during collection runs.

Characteristics

- Only available during collection runs
- Come from data files (CSV or JSON)
- Allow you to run the same request with different data sets
- Useful for data-driven testing

Setting Up Data Variables

1. Create a CSV or JSON file with your test data

   CSV Example:

   ```
   name,email,id
   John Doe,john@example.com,1001
   Jane Smith,jane@example.com,1002
   ```

   JSON Example:

   ```
   [
     {
       "name": "John Doe",
       "email": "john@example.com",
       "id": 1001
     },
     {
       "name": "Jane Smith",
       "email": "jane@example.com",
       "id": 1002
     }
   ]
   ```

2. When running a collection:

    - Click "Runner"
    - Select your collection
    - Choose your data file
    - Configure your run settings
    - Start the run

Using Data Variables

In Requests

```
https://api.example.com/users/{{id}}
```

In Scripts

```
// Get data variables in a pre-request or test script
const userName = pm.iterationData.get("name");
const userEmail = pm.iterationData.get("email");

console.log(`Processing data for ${userName} (${userEmail})`);

// Check if a data variable exists
if (pm.iterationData.has("id")) {
    console.log("User ID is available");
}
```

5. Local Variables

Local variables have the most limited scope and are only available during the current request execution.

Characteristics

- Only accessible within the current request
- Temporary and do not persist after the request completes
- Have the highest precedence (override all other variable types)
- Useful for intermediate values in scripts

Setting Local Variables

Local variables can only be set via scripts:

```
// Set a local variable
pm.variables.set("timestamp", Date.now());

// Set multiple local variables
pm.variables.set({
    "requestId": "req-" + Math.random().toString(36).substring(2, 10),
```

```
    "tempValue": "will be deleted after request"
});
```

Getting Local Variables

In the Current Request

```
https://api.example.com/log?timestamp={{timestamp}}&id={{requestId}}
```

In Scripts

```
// Get a local variable
const timestamp = pm.variables.get("timestamp");
console.log("Request timestamp:", timestamp);

// Check if a local variable exists
if (pm.variables.has("requestId")) {
    console.log("Request ID is defined");
}
```

Dynamic Variables

Postman also provides dynamic variables that are generated automatically:

- `{{$guid}}`: A v4 style GUID
- `{{$timestamp}}`: The current timestamp (Unix timestamp in seconds)
- `{{$randomInt}}`: A random integer between 0 and 1000
- And many others: `$randomEmail`, `$randomUserName`, `$randomIP`, etc.

Variable Resolution Order

When Postman encounters a variable in double curly braces `{{variable}}`, it follows this resolution order:

1. Look for a **Local variable** with that name
2. Look for a **Data variable** with that name (during collection runs)
3. Look for an **Environment variable** with that name (in the active environment)
4. Look for a **Collection variable** with that name
5. Look for a **Global variable** with that name
6. If not found in any scope, the variable doesn't get resolved

Best Practices for Postman Variables

1. **Choose the appropriate scope**: Use the narrowest scope needed for your variables
    - Global: Only for values used across all collections
    - Collection: For values specific to a collection

    - Environment: For values that change between environments
    - Local: For temporary values

2. **Use descriptive names**: Follow a naming convention that makes the purpose clear
    - Bad: `url`, `key`
    - Good: `baseApiUrl`, `authToken`

3. **Secure sensitive data**: Use environment variables for credentials
    - Mark sensitive values as "secret" in the UI
    - Never hardcode credentials in collection scripts
    - Consider using Postman's Secrets feature for production credentials

4. **Document your variables**: Add descriptions to variables in the UI
    - Helps team members understand their purpose
    - Include example values where helpful

5. **Maintain variable hygiene**:
    - Remove variables when they're no longer needed
    - Regularly audit global and environment variables
    - Clear temporary variables after use